LAB 15

Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Backtest engine
Duration	30 min
Simulator page	http://localhost:8000/#/backtest?strategy=dip_buy&symbols=SPY&costs=zero
Mode	No coding. Runs ONLINE with your Alpaca paper keys (Integrations page), or OFFLINE on the demo data and the built-in paper broker. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Alpaca advertises commission-free trading and a trader concludes that costs do not matter. Before the pipeline is automated, the head of execution wants the backtest engine to show gross versus net results under four cost scenarios, where the money goes, the break-even cost of a fast and a slow strategy, and how sensitive results are to the assumed fill time.

Objective

Run the same strategy under four cost models and find its break-even cost.

Before You Start

- Make sure the Trading Simulator is running and open http://localhost:8000 > Course Labs > Lab 15. It is installed once - see the guide 'Download, Install and Start the Trading Simulator Software' in the Learner Guide.
- Record every result you are asked to check - you will use it in the debrief and in the later labs.

Lab Steps - In the Trading Simulator

No coding is needed. Make sure the Trading Simulator is running (installed once - see the guide 'Download, Install and Start the Trading Simulator Software') and open http://localhost:8000. In the sidebar open Course Labs > Lab 15: the app shows the same steps with a Go button that opens each page pre-filled. Every step works offline on the bundled demo data and the built-in paper broker.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1. Zero vs Alpaca vs high-cost

Run the same strategy three times, changing ONLY the Cost model, to see how much of its profit trading costs eat. The link opens Dip buy after a losing streak on SPY (n down 3, hold 3) with Zero cost chosen. In the left panel, under Costs & execution, pick each cost model below in turn and press Run backtest after each one. After each run open the 'Costs: gross vs net' tab on the right and write down three rows: total, gross CAGR -> net CAGR, and cost drag (CAGR). Every run is kept in the History card, so you can reopen it.

- **Run 1 - Cost model:** Zero cost (zero): no costs at all, so net = gross. This is the strategy's raw edge.
- **Run 2 - Cost model:** Alpaca commission-free (alpaca_retail): no commission, but you still pay half the 1 bp spread + 1 bp slippage = about 1.5 bps per trade, plus small regulatory fees on sells.
- **Run 3 - Cost model:** High-cost route (high_cost): 10 bps commission + half the 5 bp spread + 5 bps slippage = about 17.5 bps per trade - an expensive retail or offshore broker.
- **Everything else:** leave as opened - Execution next bar open, Benchmark SPY, Start and End blank (all the demo data)
- **What a bp is:** 1 basis point (bp) = 0.01%, so 17.5 bps on a $10,000 trade costs $17.50

Open: http://localhost:8000/#/backtest?strategy=dip_buy&symbols=SPY&costs=zero (Backtest SPY dip-buy, zero cost first)

Check: About 102 trades (203 fills) in every run. Net CAGR falls from 5.09% (zero) to 4.65% (alpaca_retail, about $3,700 of costs) to 0.48% (high_cost, about $33,600 - a third of the $100,000 capital - a cost drag of 4.61% a year). Dip-buy trades often, so every extra bp is paid about 200 times.

Step 2. Break-even and execution timing

Break-even cost is the one-way cost per trade at which the strategy's net CAGR falls to zero: above it the strategy loses money. It depends only on the strategy, so it is the same in all three Step 1 runs. Read it in the BREAK-EVEN COST tile (top right) or the 'break-even one-way cost' row of the 'Costs: gross vs net' tab, and compare it with the cost per trade of each route in Step 1. Then test the execution assumption: the link opens the high-cost run again with Execution = next bar close. Press Run backtest and compare it with your Step 1 high-cost run.

- **Execution: next bar open:** the default (like TradingView): the signal is decided at today's close and the order fills at tomorrow's open
- **Execution: next bar close:** the same signal fills at tomorrow's close instead - a full day later
- **Cost model:** High-cost route (high_cost) - keep it, so only Execution changes
- **Compare:** CAGR, Sharpe, the BREAK-EVEN COST tile and gross CAGR -> net CAGR

Open: http://localhost:8000/#/backtest?strategy=dip_buy&symbols=SPY&costs=high_cost&execution=next_close

Check: Break-even is 19.5 bps. Alpaca's ~1.5 bps leaves plenty of room, but the high-cost route's ~17.5 bps (plus fees) uses almost all of it - that is why its net CAGR was only 0.48%. With next bar close, gross CAGR moves from 5.09% to 7.67%, net CAGR from 0.48% to 2.95% and break-even from 19.5 to 29.0 bps. One fill-timing assumption moved the result that much: a strategy whose break-even is only a few bps, or that depends on the fill assumption, is fragile.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3. Stops and sizing

Now add risk controls to a different strategy, EMA crossover 20/50 on SPY. The link fills in the settings below; click 'position sizing, stop-loss, take-profit...' under Costs & execution to see them. Press Run backtest and open the Trade list tab: the Exit reason column says why each trade closed. Then clear Stop-loss %, set Position sizing back to % of equity (Size 1.0), run again and compare the two.

- **Strategy:** EMA crossover - fast 20, slow 50, symbol SPY
- **Cost model:** Alpaca commission-free (alpaca_retail)
- **Position sizing:** volatility target - Target vol (ann.) 0.15, Max leverage 1.0: the position is scaled to 0.15 / SPY's recent volatility (capped at 100% of equity), so it shrinks when the market is volatile
- **Stop-loss %:** 5 - a long position is sold if the price falls 5% below the entry price; the strategy then waits for a fresh signal before buying again
- **Take-profit %:** leave blank

Open: http://localhost:8000/#/backtest?strategy=ema_cross&symbols=SPY&fast=20&slow=50&stop_loss_pct=5&sizing=vol_target

Check: 16 trades: 4 exit with stop_loss, the rest with signal. With the stop and vol-target sizing the strategy is invested about 56% of the time (exposure) against 74% without them, but CAGR is 3.0% against 8.8% and the max drawdown is no better (-29.8% against -25.5%). Risk controls are not free: backtest them, never assume they help.

Expected in the app: Gross vs net CAGR, a cost breakdown and the break-even bps.

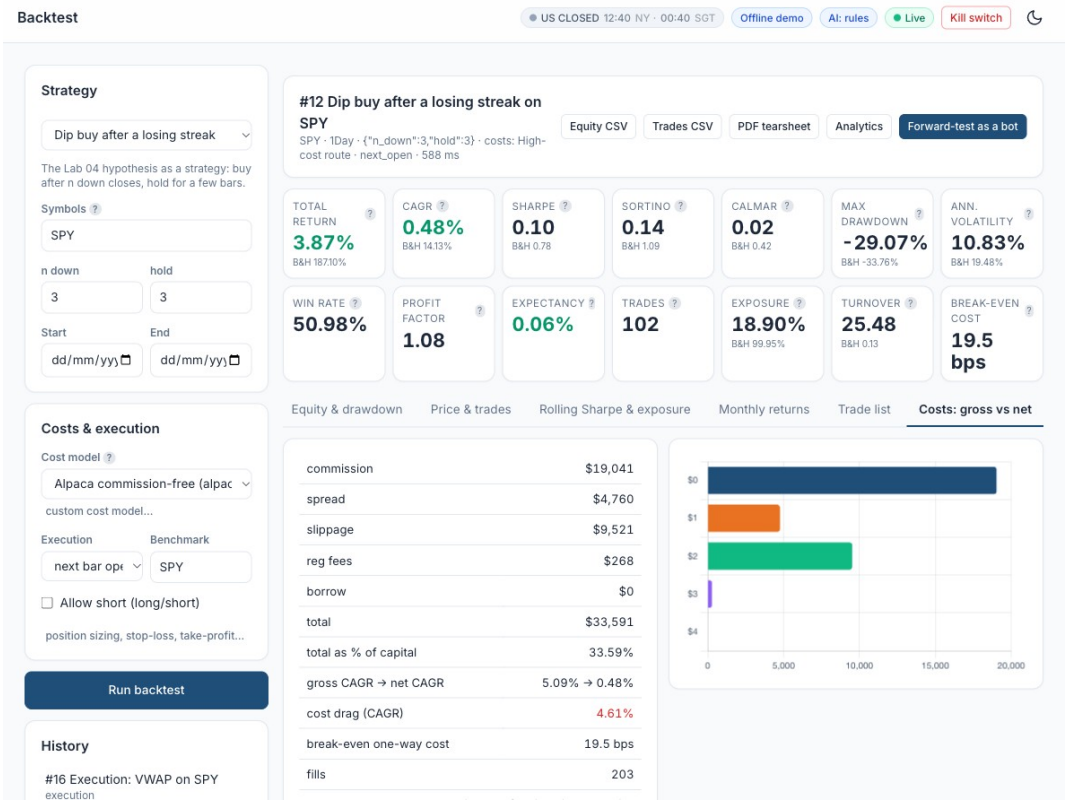


Figure: Lab 15 in the Trading Simulator.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Optional: Let AI Write the Code

This lab needs no coding. To see, run or customise the code behind it, click a link: the AI opens with the prompt filled in and writes the code for you. All prompts are also in labs/Prompt_Library.html and .pdf.

Trading costs and the break-even cost - run it in Google Colab
I am a beginner and do not write code. Write ONE complete Python script that runs as-is in a Google Colab notebook. Start with a !pip install line for any library Colab lacks, use pandas, numpy, matplotlib, scipy, scikit-learn, statsmodels and yfinance only, get daily prices from yfinance (auto_adjust=True unless I say otherwise), put a one-line comment above each section, print results as clear tables and charts, and after the code explain in 5 short bullet points how to read the output. Paper trading and education only. Task: backtest 'buy when RSI(2) < 10, sell when RSI(2) > 70' on SPY 2018-2025 at the next day's open under three cost models - zero, retail (0.02% spread + 0.01% slippage) and high (0.10% + 0.05%). Print gross and net CAGR and Sharpe for each, find the break-even one-way cost in basis points by bisection, and compare next-day-open with next-day-close execution.

Open in Claude | **Open in ChatGPT** | **Open Google Colab**

Debrief

Commission is only one trading cost: spread and slippage scale with turnover, and the assumed fill time can matter as much as the fee schedule. Coding the cost model into the engine and testing outcomes under several scenarios shows whether an edge survives real-world execution - the break-even cost is the single number that says how much room the strategy has.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.